Republic Of Cameroon

Peace-Work-Fatherland

PK Fokam Institute of Excellence

République Du Cameroun

Paix-Travail-Patrie

PK Fokam Institute of Excellence

Project: Wedding Invitation and Gift Management System (WIGMS)

Third Progressive Report

V1.2

GROUP 1		
NAME	MATRICULE	PERCENTAGE PARTICIPATION
TCHOUNGA NGATCHOU MARIE CLAIRE	26S022	18.5%
MASSADO DJOKO AICHA CLAUDIA	26S033	18.5%
NKOMPNANG GUILLAUME JUNIOR	26S017	18.5%
AVOUTIA DONGMO BRAYANN	26S028	18.5%
NZANUM RYAN DZEMBUIN	26S007	13%
Chi Rawlings Ngang	25S021	13%

SUPERVISOR: MR MBIETIEU AMOS

Academic Year: 2025/2026

Table of Contents

1. Introduction…….3

2. Gantt Chart ……..3

3. System Design……3

4. Sequence diagram……4

5. Deployment diagram……6

6. Class diagram……6

7. Work distribution……7

8. Module One……8

9. GitHub Repository……12

10. Module Two……..16

11. Repository Link……17

12. Graphic User Interface……18

13. Tools And Technologies used……..24

14. Difficulties encountered and Proposed Solutions……...24

15. Next steps of the project……..25

16. Conclusion……25

1. Introduction

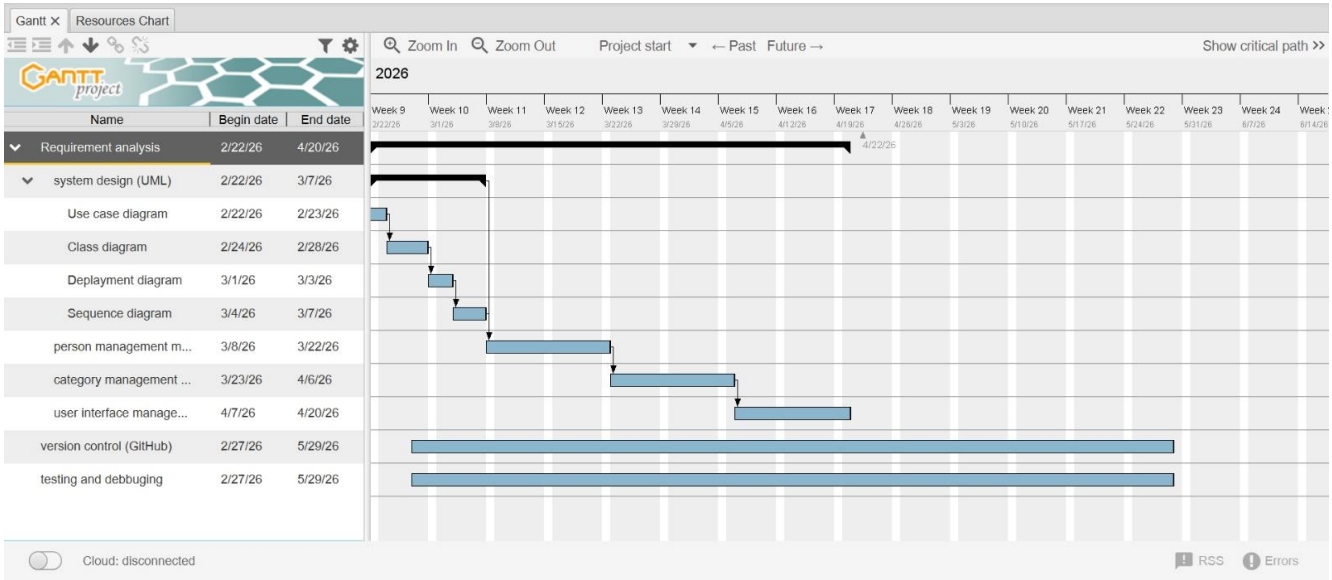
The aim of this project is to design and implement a Wedding Invitation and Gift Management System using the C programming language. The system is developed using a modular approach, allowing each component to be built, tested, and improved independently.

This second report presents:

- The corrected version of Module 1: Person Management based on supervisor remarks
- The implementation of Module 2: Category Management

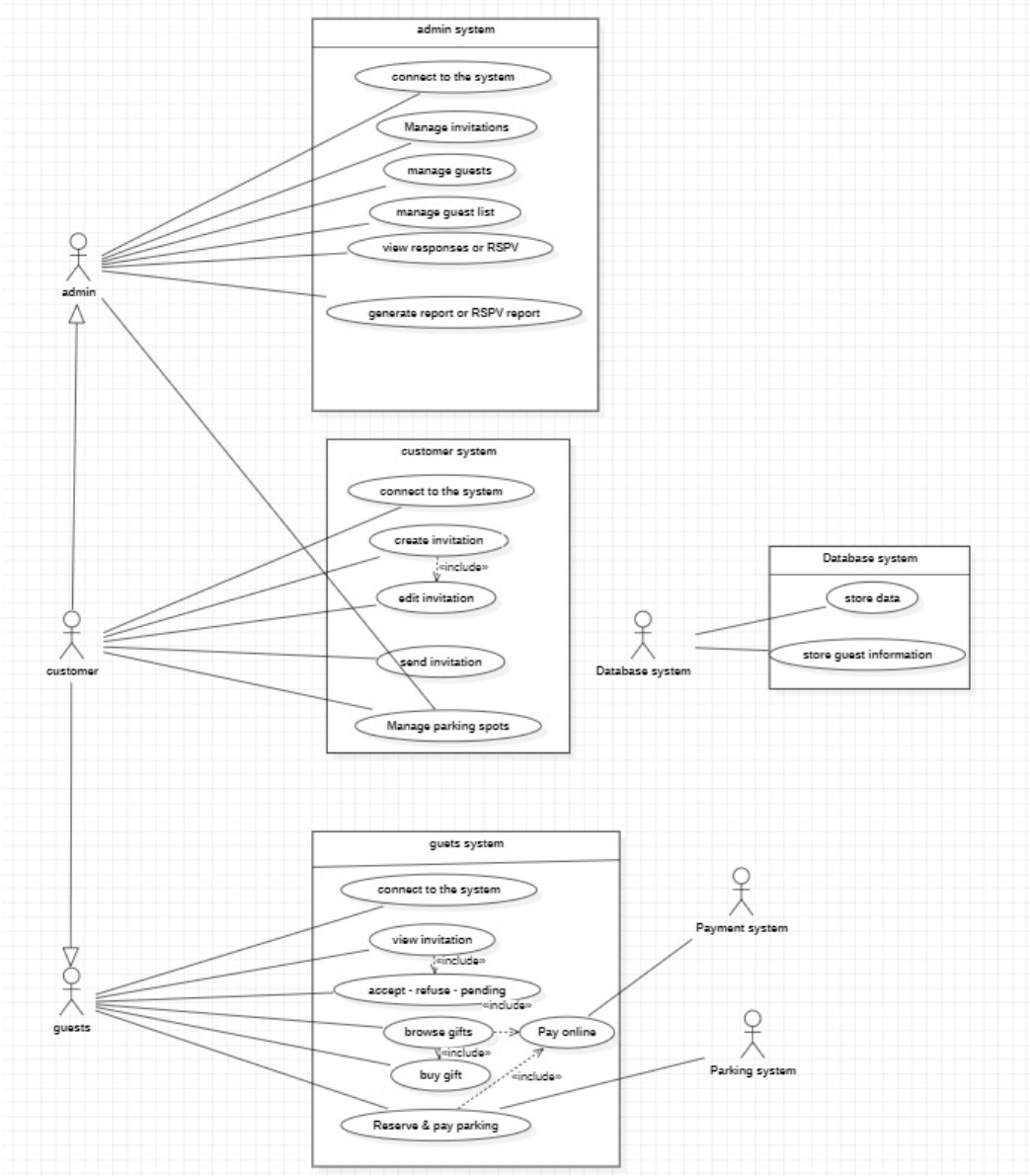
Special attention was given to improving the user interface, ensuring better usability before proceeding with advanced modules.

2. Gantt Chart



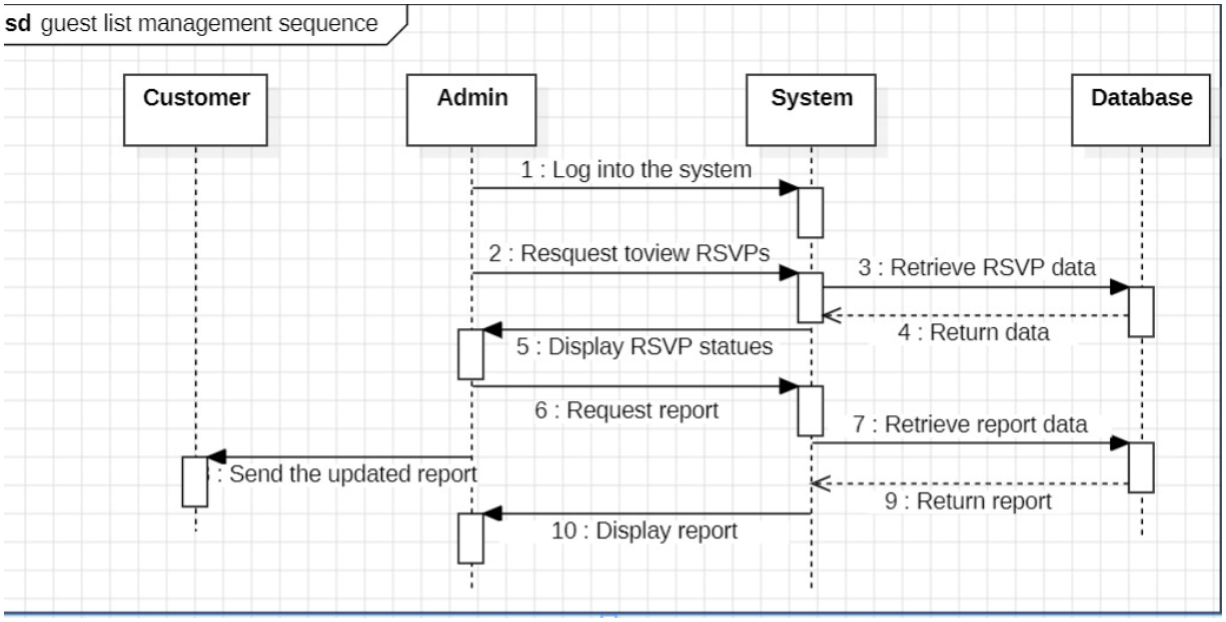
3. System Design

3.1 Use Case Diagram

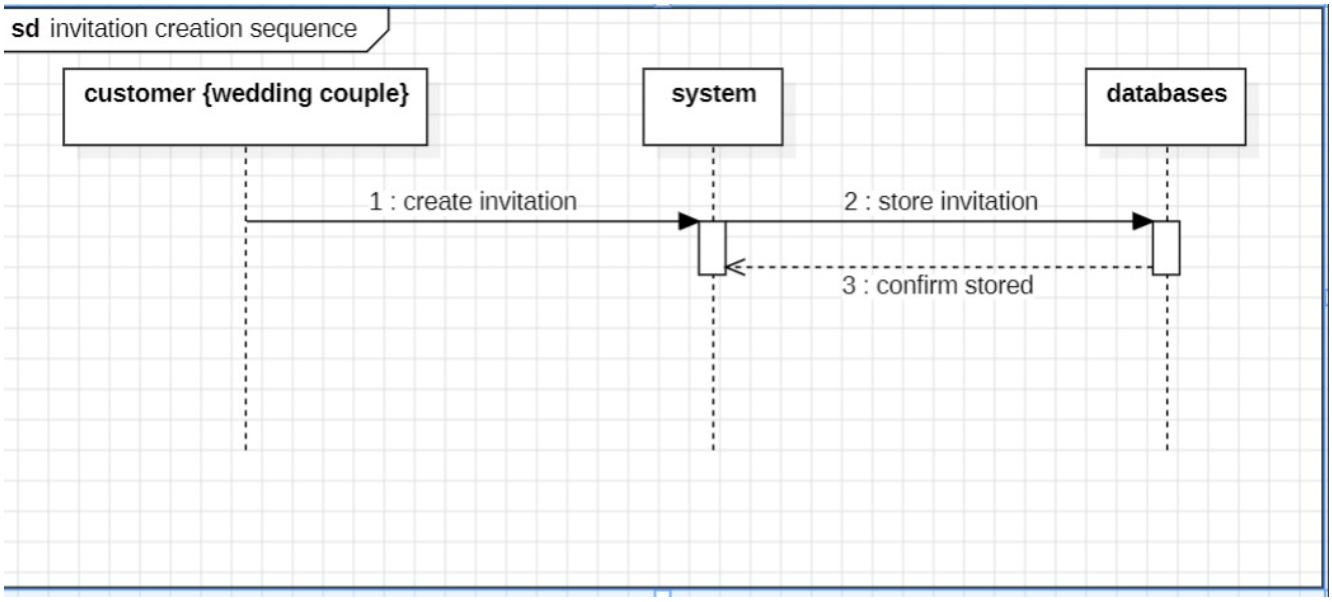


3.2 Sequence Diagram

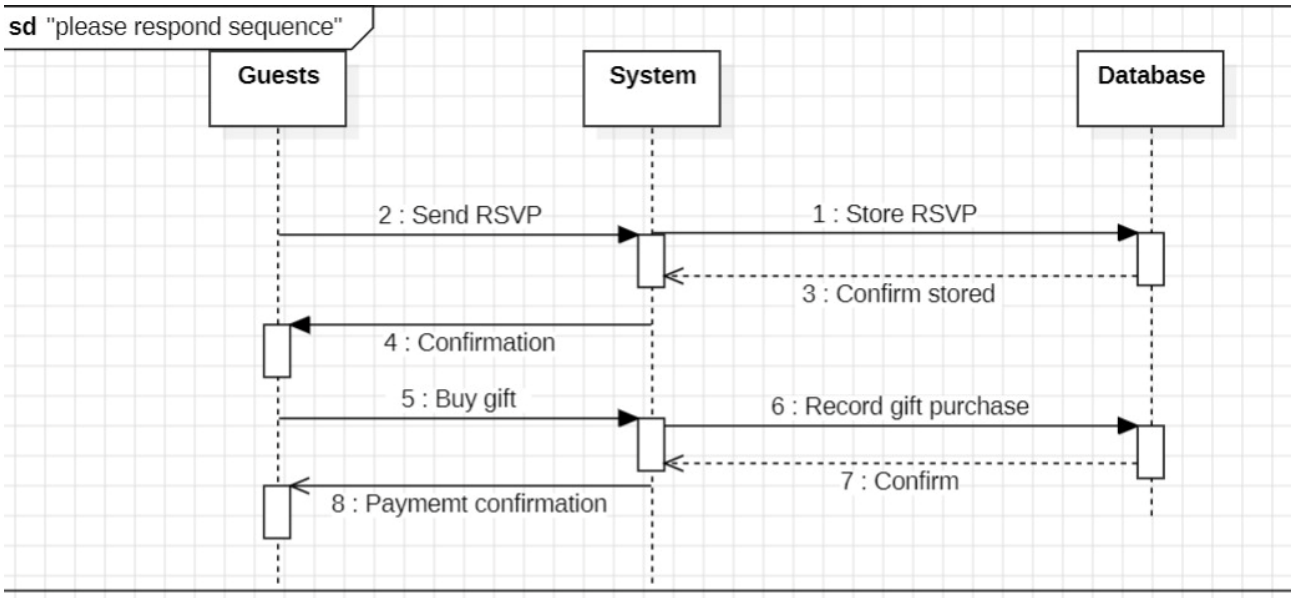
3.2.1 Guest list management sequence



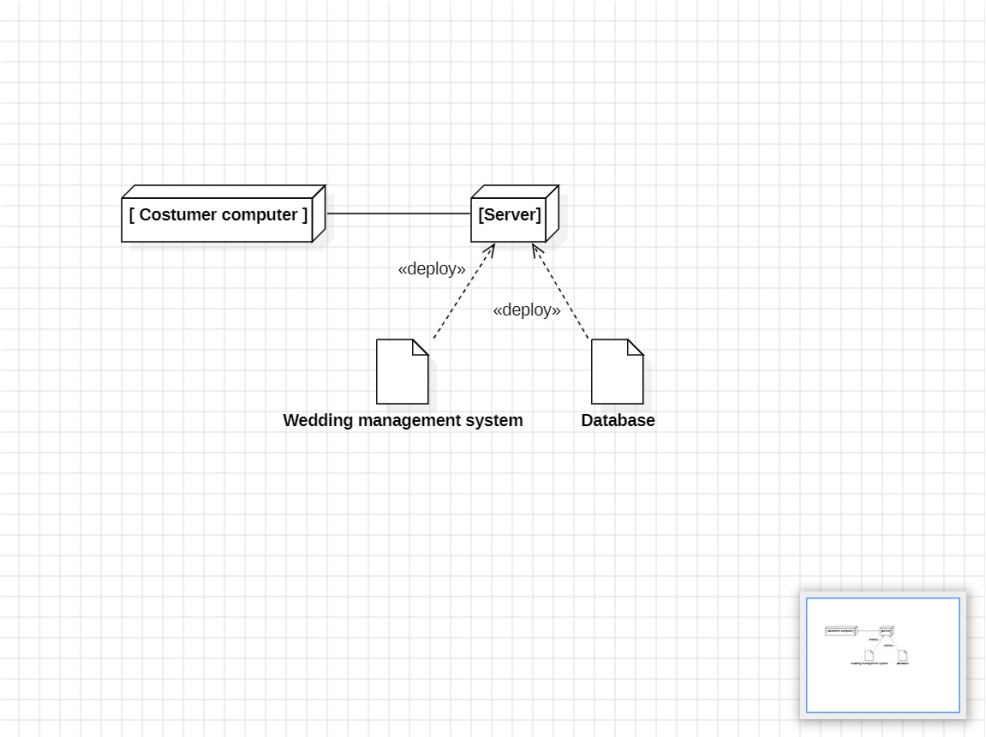
3.2.2 Invitation creation sequence



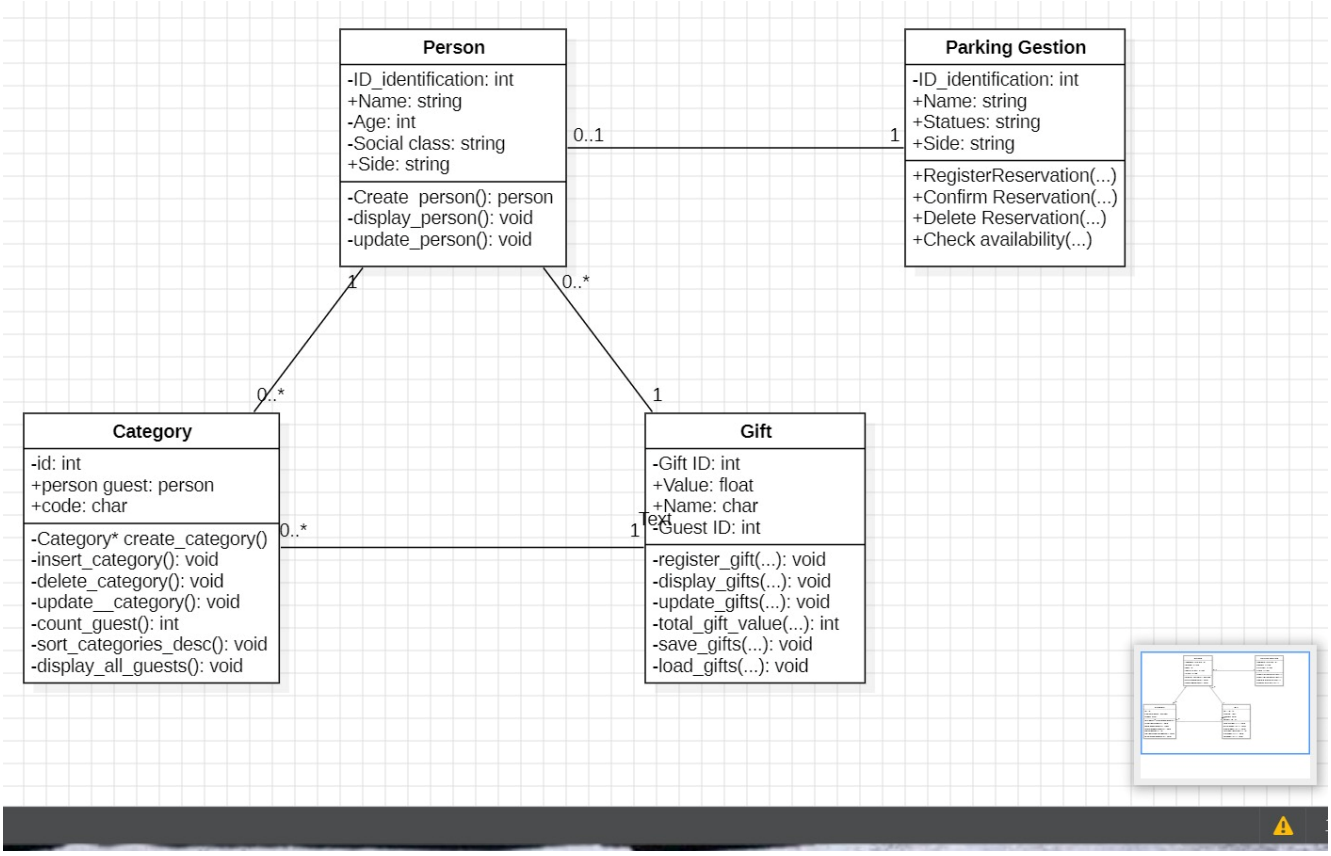
3.2.3 Respond sequence



4. Deployment Diagram



5. Class Diagram



6. Work Distribution

Group Member	Role	Assigned Tasks
Tchounga Ngatchou Marie Claire	Project Manager and Coordinator	Overall project coordination and writing and formatting report
Avoutia Dongmo Brayann	Module 1 & 2 developer	Implementation of all libraries and debugging module 1 &2
Nkompnang Guillaume Junior	Module 1 & 2 designer	Graphical user interface design
Massado Djocko Aicha	System and Gantt chart designer	Case and sequence diagram design.
Nzanum Ryan Dzembuin	Data structure and algorithm	Implementation of delete category
Chi Rawlings Ngang	Documentation and GitHub manager	Managing GitHub repository

7. User Interface

To improve interaction with the system, a simple user interface was introduced using standard input/output functions in C. The interface allows users to navigate through the program using a menu-driven approach.

```c
#ifndef TYPES_H
#define TYPES_H

typedef enum{
    LE,
    LA
}side;

typedef struct{
    int id;
    char name[100];
    int age;
    char social_class[50];
    side side;
}Person;

typedef struct category{
    int id;
    char code[50];
    Person guest[4];
    struct category *next;
}Category;

typedef struct{
    int gift_id;
    char name[100];
    float value;
    int guest_id;
}Gift;

#endif
```

MODULE 1: PERSON MANAGEMENT

1. Objectives of the Module

- Define a Person structure to store guest information.
- Implement a function to create a new person.
- Implement a function to display a person's information.
- Implement a function to update a person's information.
- Apply structured programming concepts using C language.

2. Person Structure

A Person represents an invited guest. The structure stores basic information such as ID, name, age, social_class.

```c
typedef struct{
    int id;
    char name[100];
    int age;
    char social_class[50];
    side side;
}Person;
```

3. Implemented Functions

a.1 create_person ()

```c
Person create_person(){
    Person P;
    printf("\n- - - Guest Input - - -\n");
    printf("ENter an ID\n");
    scanf("%d",&P.id);
    getchar();

    printf("\n- - - Guest Name - - -\n");
    printf("Enter a name\n");
    fgets(P.name,sizeof(P.name),stdin);


    printf("\n- - - Guest AGE - - -\n");
    printf("ENter your Age\n");
    scanf("%d",&P.age);
    getchar();

    printf("\n- - - Guest Social Class - - -\n");
    printf("Enter your Social Class\n");
    fgets(P.social_class,sizeof(P.social_class),stdin);

    printf("\n - - -Guest side- - -\n");
    printf("Enter 0 for LE\n");
    printf("Enter 1 for LA\n");
    scanf("%d",&P.side);
    return P;
```

a.2 Create_Person() (Parking)

```c
    printf("\n - - - PARKING - - - \n");

    if (strcmp(P.social_class, "VIP") == 0) {

        do {
            printf("VIP Access: Enter your preferred parking_id (1-100): ");
            scanf("%d", &parking_id);

        } while (1);
        A[0] = parking_id;
        printf("ID %d assigned to VIP.\n", parking_id);
    } else {

        int random_id;
        int slot_found = -1;
        for (int i = 0; i < 100; i++) {
            if (A[i] == 0) {
                slot_found = i;
                break;
            }
    }
        do {
            is_taken = 0;
            random_id = (rand() % 900) + 100;

            for (int i = 0; i < 100; i++) {
                if (A[i] == random_id) {
                    is_taken = 1;
                    break;
                }
            }
        } while (is_taken);

        if (slot_found != -1) {
            A[slot_found] = random_id;
            printf("Standard Access: You have been assigned ID: %d\n", random_id);
        }
    }

    return P;
}
```

b. display_person ()

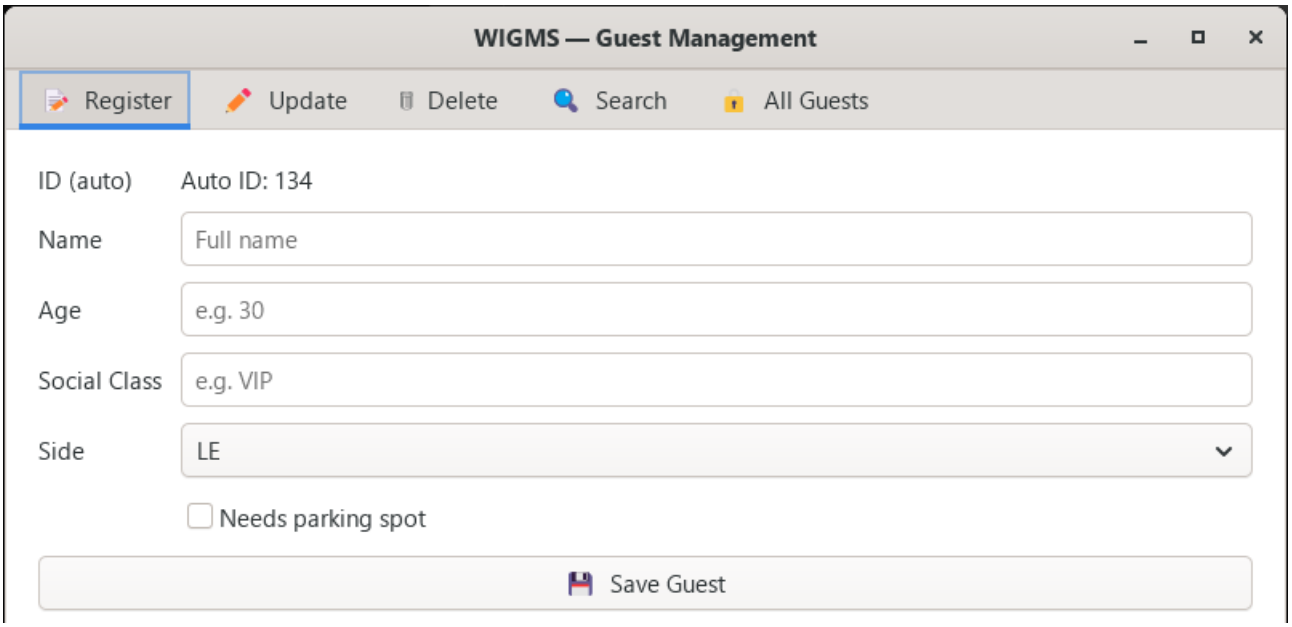
```c
void display_Person(Person P)
{
    int side_choice;
    printf("\n- - - Guest details- - -\n");
    printf("ID:%d\t | Age:%d\t |",P.id,P.age);
    printf("Parking_ID:%d",P.parking_id);

    if(P.side==0)
    {
        printf("side 'LE'\t");
    }
    else
    {
        printf("side 'LA'\t");
    }
    printf("| Name:%s | Social_class:%s ",P.name,P.social_class);
}
```

4. Graphic User Interface

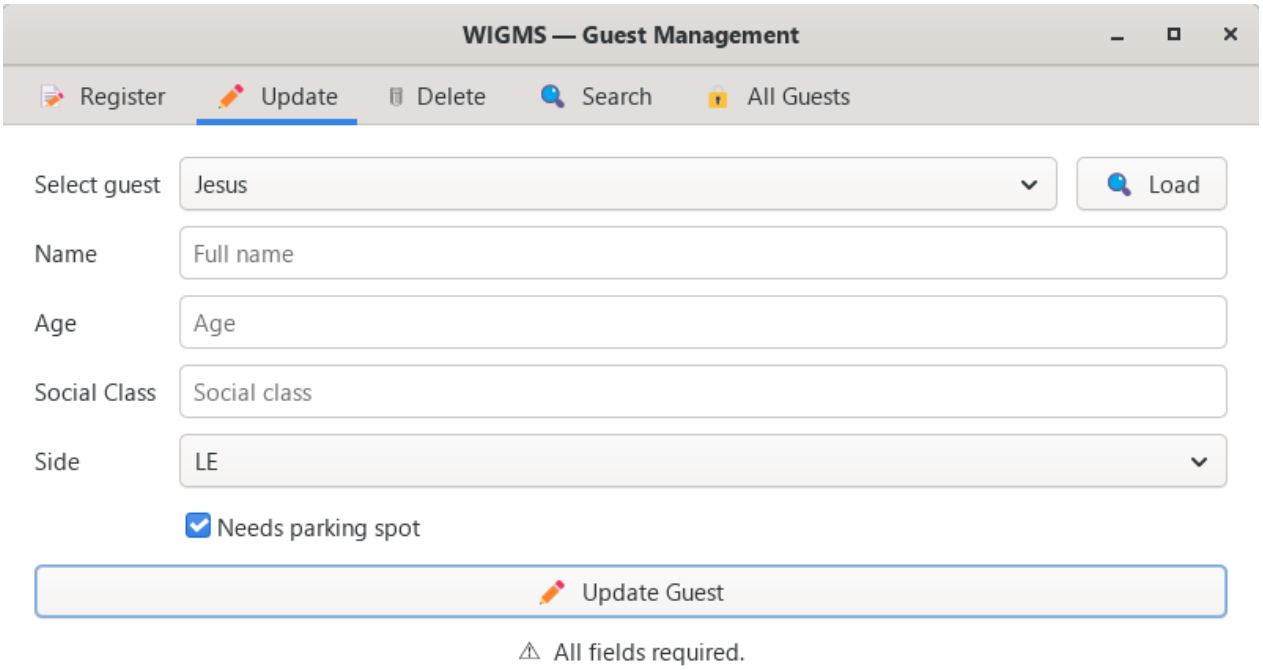
Person Module for Graphic User Interface

: For the person module graphic user interface, we are handling the regidtration of the different guests that will participate to the event, the modification of their entered paramaters and the suppression of those whose turned down their invitation. In this program, we can do a lot of tasks like;

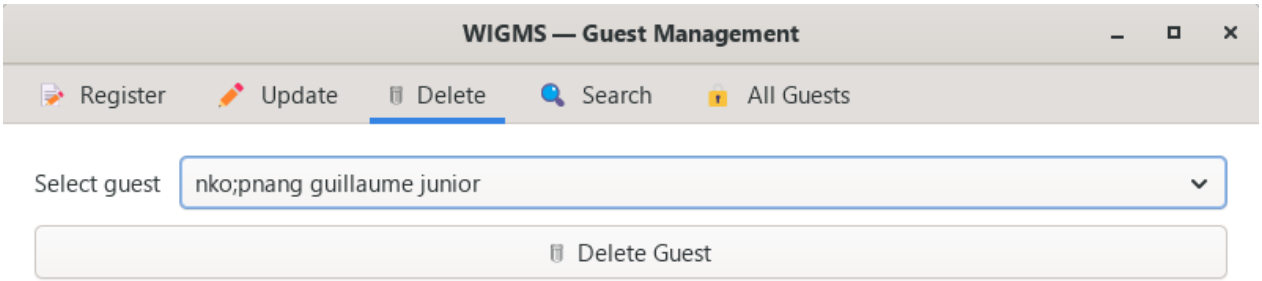
- **The Registration:** here, we are registering the different guests which would like to attend to the ceremony. The user will have to enter his/her entire name, age and social class. We also made ourselves sure to let the ID of each person be automatically generated and to leave a check box to say if he need a parking spot or not. It is as we can see here below:



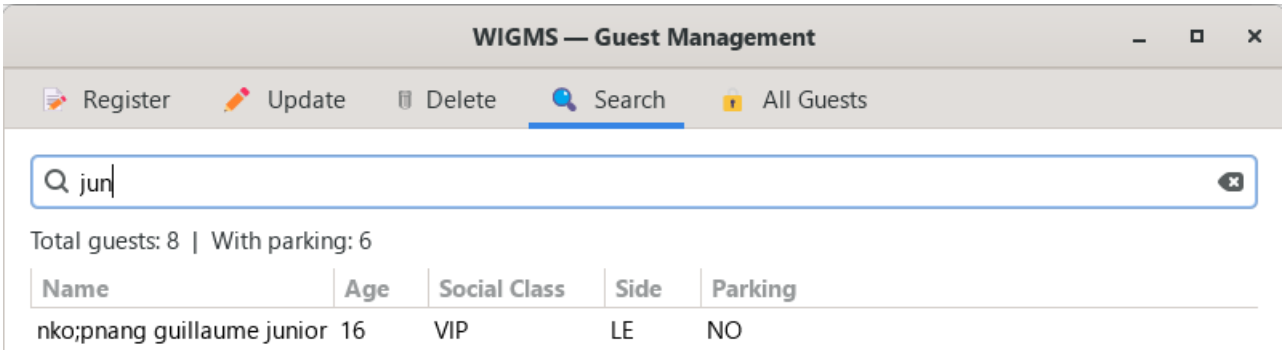
- **Update:** here we have the ability to modify the information on each guest just by entering the name of the of the person in the research space and just modifying the information which are already inside the person description. Like here:



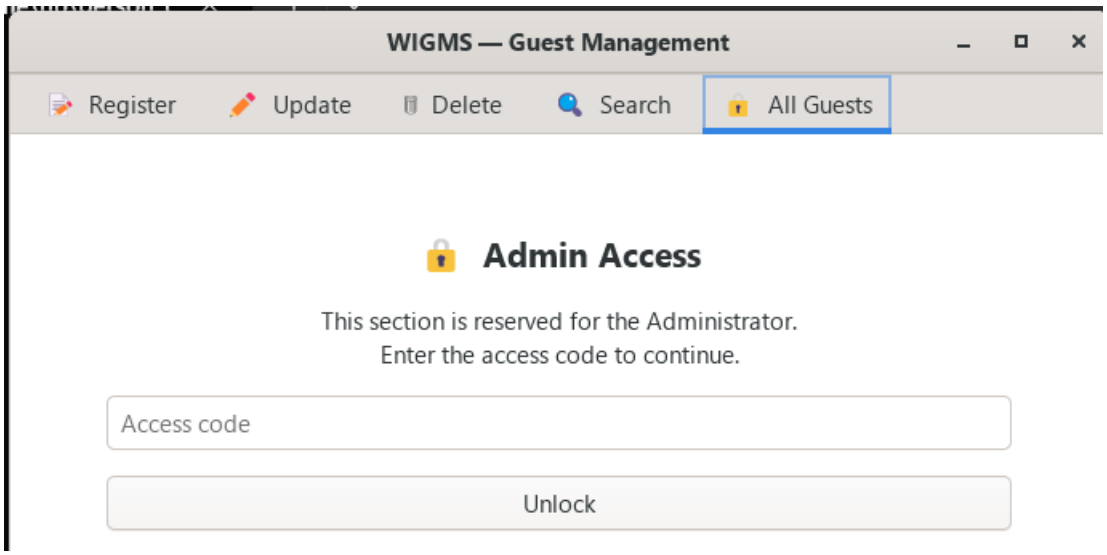
- **Delete: :** It's an option where it's possible for an user to delete an existant guest and the informations about him in the case that you have created a person which where not supposed to exist. It's like on this picture, where you just have the charge of write the name of the person and select delete.

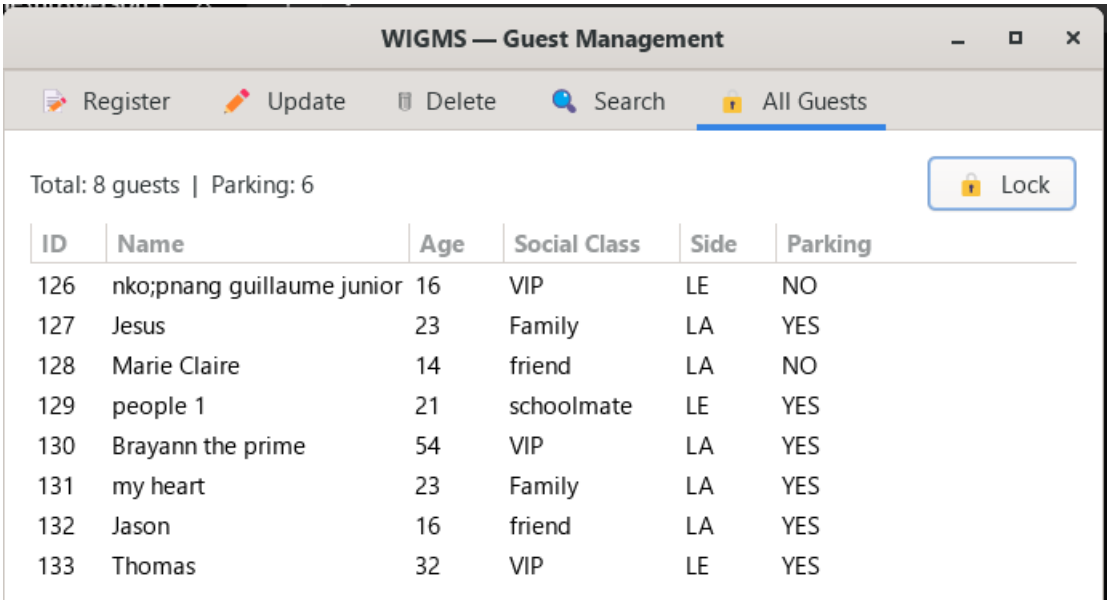


- **Search:** Here is an option where you can look for somebody in the list if guest by looking for his name or his social class in the search bar. It can be use to group the people by social class or by name.



- **All guests:** this class give to a user the access to all the guests which were registered in the program, the number of Guests which were stocked in the program, the ID of all the guests that where created in the application and the total number of parking spot assigned to guests. Since the ID and the information about the different Guests are confidential, we thought that it will be wise to protect them with a password which is "pokemon" in order to assure the protection of their data. The presentation of this page is like this:





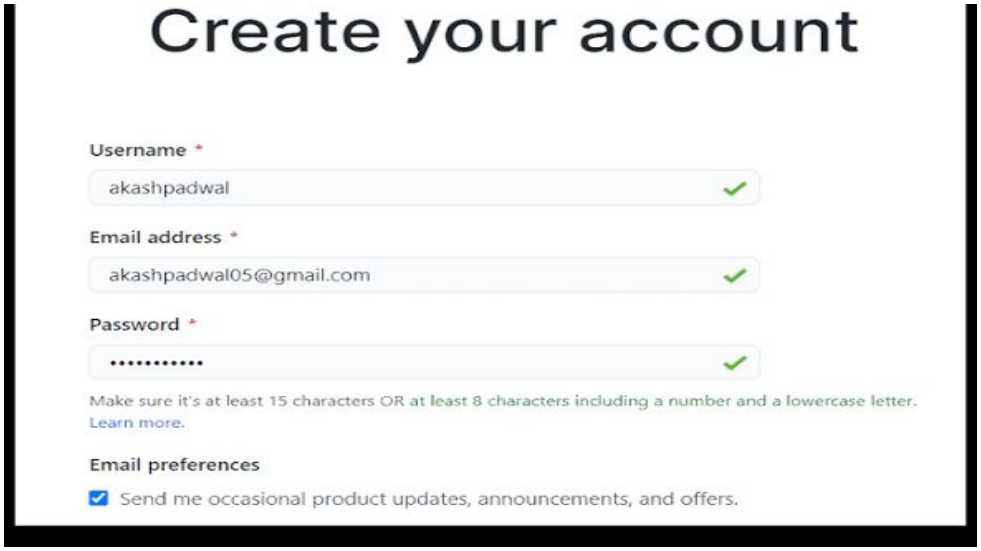
ID	Name	Age	Social Class	Side	Parking
126	nko;pnang guillaume junior	16	VIP	LE	NO
127	Jesus	23	Family	LA	YES
128	Marie Claire	14	friend	LA	NO
129	people 1	21	schoolmate	LE	YES
130	Brayann the prime	54	VIP	LA	YES
131	my heart	23	Family	LA	YES
132	Jason	16	friend	LA	YES
133	Thomas	32	VIP	LE	YES

5. Setting Up a GitHub Repository

To create a GitHub repository using the web interface, follow these steps:

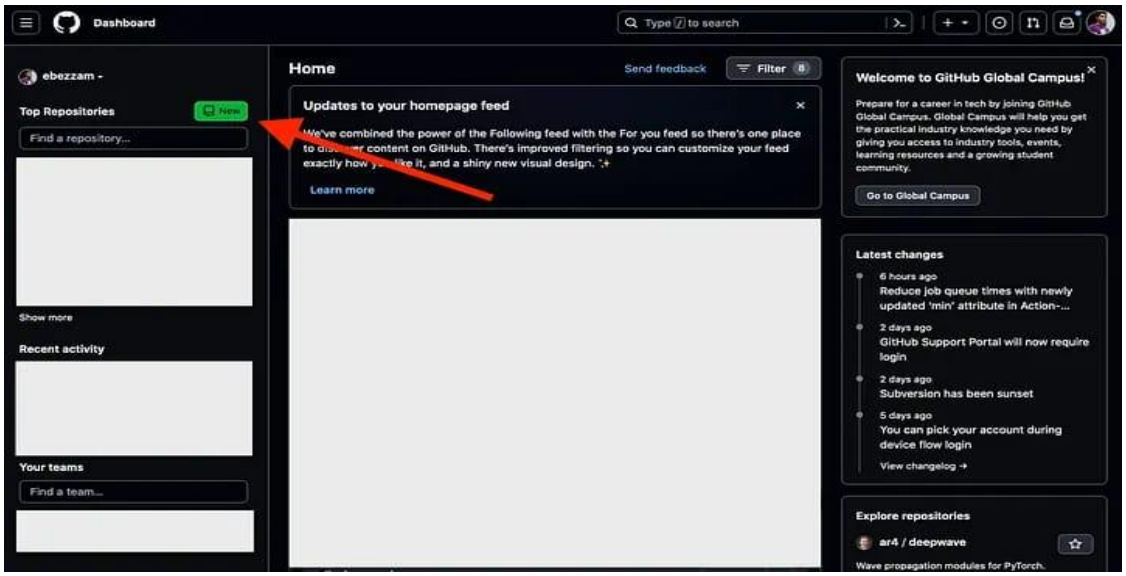
Step 1: Creating the GitHub accounts

To sign up for an account, navigate to https://github.com/ and follow the prompts. To keep your GitHub account secure you should use a strong and unique password.



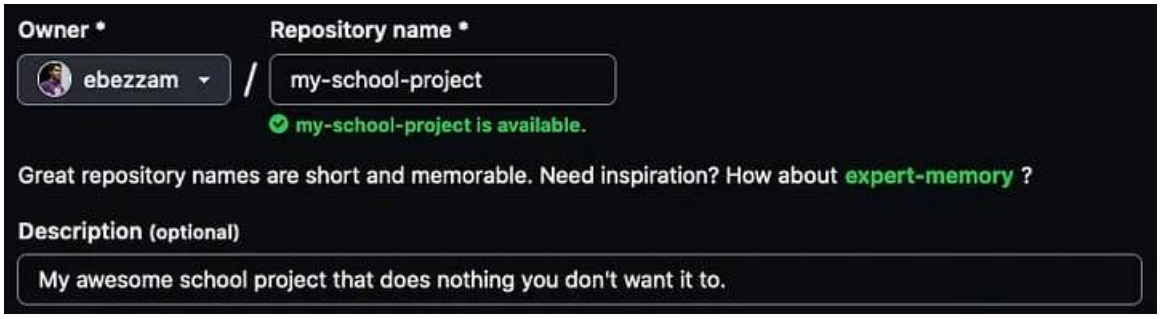
Step 2: Creating the Repository

Start a new repository in the upper-right corner of any page, select the plus (+) icon, then click New repository



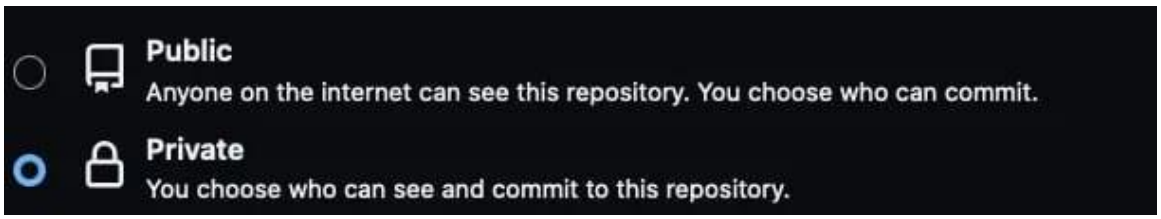
Step 3: Specify an owner

If you belong to any organizations, use the "Owner" dropdown menu to select the account (your personal account or an organization) that will own the repository. Enter a name — type a short, memorable name for your repository in the "Repository name" box.



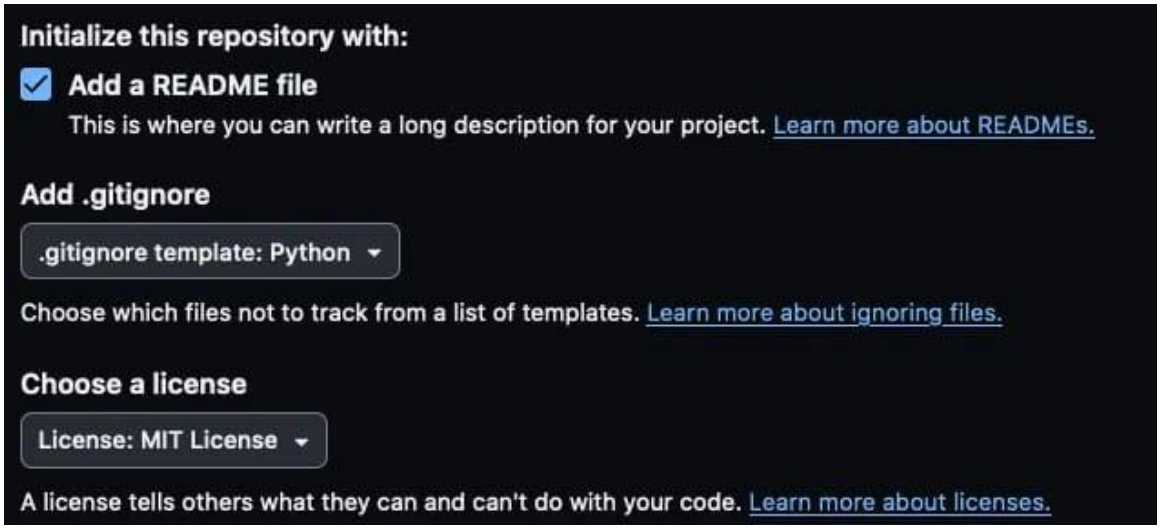
Step 4: Choose repository visibility

Decide whether the repository will be Public (anyone can see it) or Private (only you and collaborators you choose can access it).



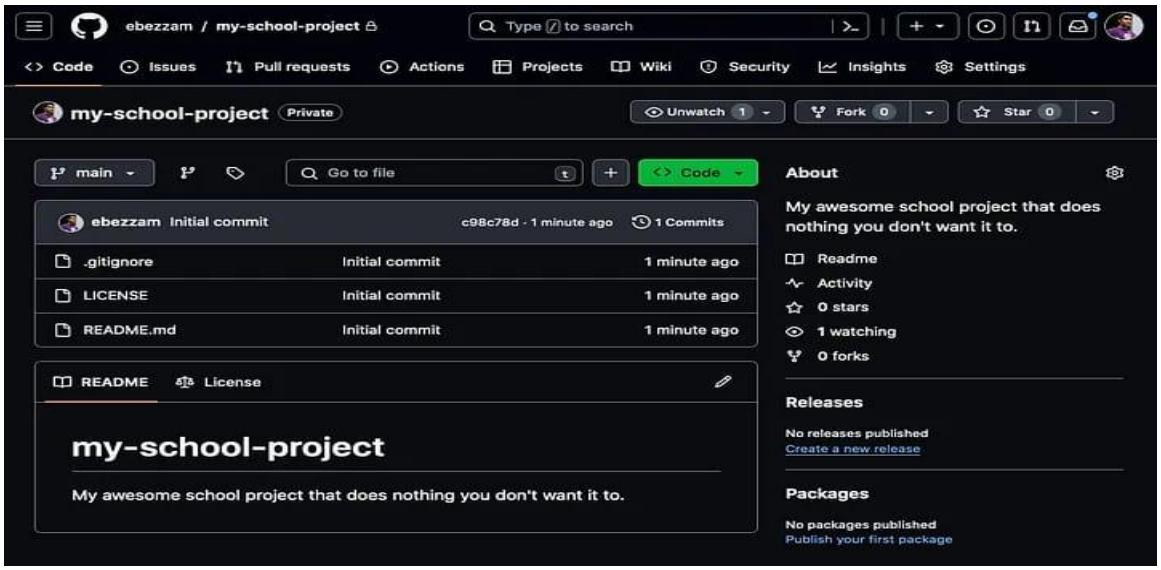
Step 5: Initialize the repository

Select Add a README file to create a document that describes your project. You can also optionally add a .gitignore file (to exclude specific file types from version control) and a license.



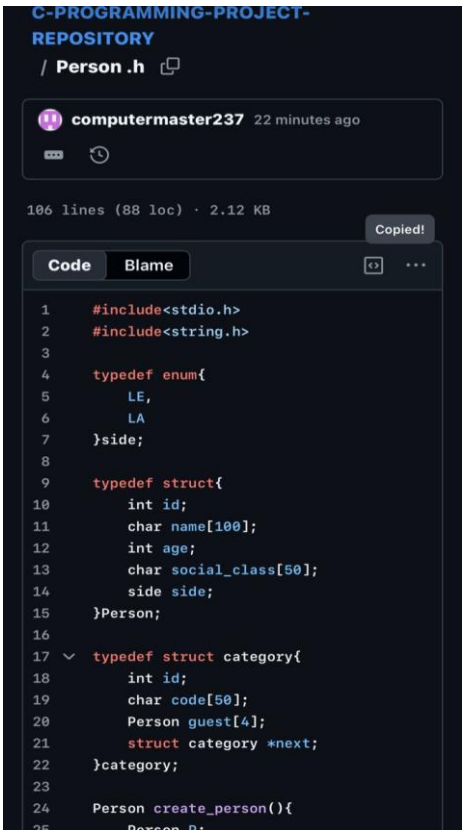
Step 6: Create the repository

Click the green Create repository button at the bottom of the page.



Step 7: Start using your repository

GitHub repository is now created and ready for you to start adding files and managing your project with version control. You can then clone the repository to your local machine using the provided URL to work with it locally.



MODULE 2: CATEGORY MANAGEMENT

1. Objectives

- Organize guests into categories
- Use linked lists for dynamic storage
- Improve system structure

2. Category Structure

```c
typedef struct category{
    int id;
    char code[50];
    Person guest[4];
    struct category *next;
}Category;
```

3. Implemented Functions

3.1 Create_Category

```c
Category* create_category() {
    Category* new_node = (Category*)malloc(sizeof(Category));
    if (new_node == NULL) return NULL;

    printf("Enter Category ID: ");
    scanf("%d", &new_node->id);
    printf("Enter Category Code: ");
    scanf("%s", new_node->code);

    new_node->next = NULL;
    return new_node;
}

void insert_category(Category **tete) {
    Category* new_cat = create_category();
    if (new_cat) {
        new_cat->next = *tete;
        *tete = new_cat;
    }
}
```

3.2 Update_category

```c
void update_category(Category *tete, int id) {
    Category *current = tete;
    while (current != NULL) {
        if (current->id == id) {
            printf("Enter New Code: ");
            scanf("%s", current->code);
            return;
        }
        current = current->next;
    }
}
```

3.3 Delete_category

```c
void delete_category(Category **tete, int id) {
    Category *temp = *tete, *prev = NULL;
    if (temp != NULL && temp->id == id) {
        *tete = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->id != id) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);
}
```

4. Linked List Explanation

Categories are stored using a linked list:

- Each node represents a category
- Nodes are connected using pointers
- Allows dynamic insertion and deletion

5. Repository

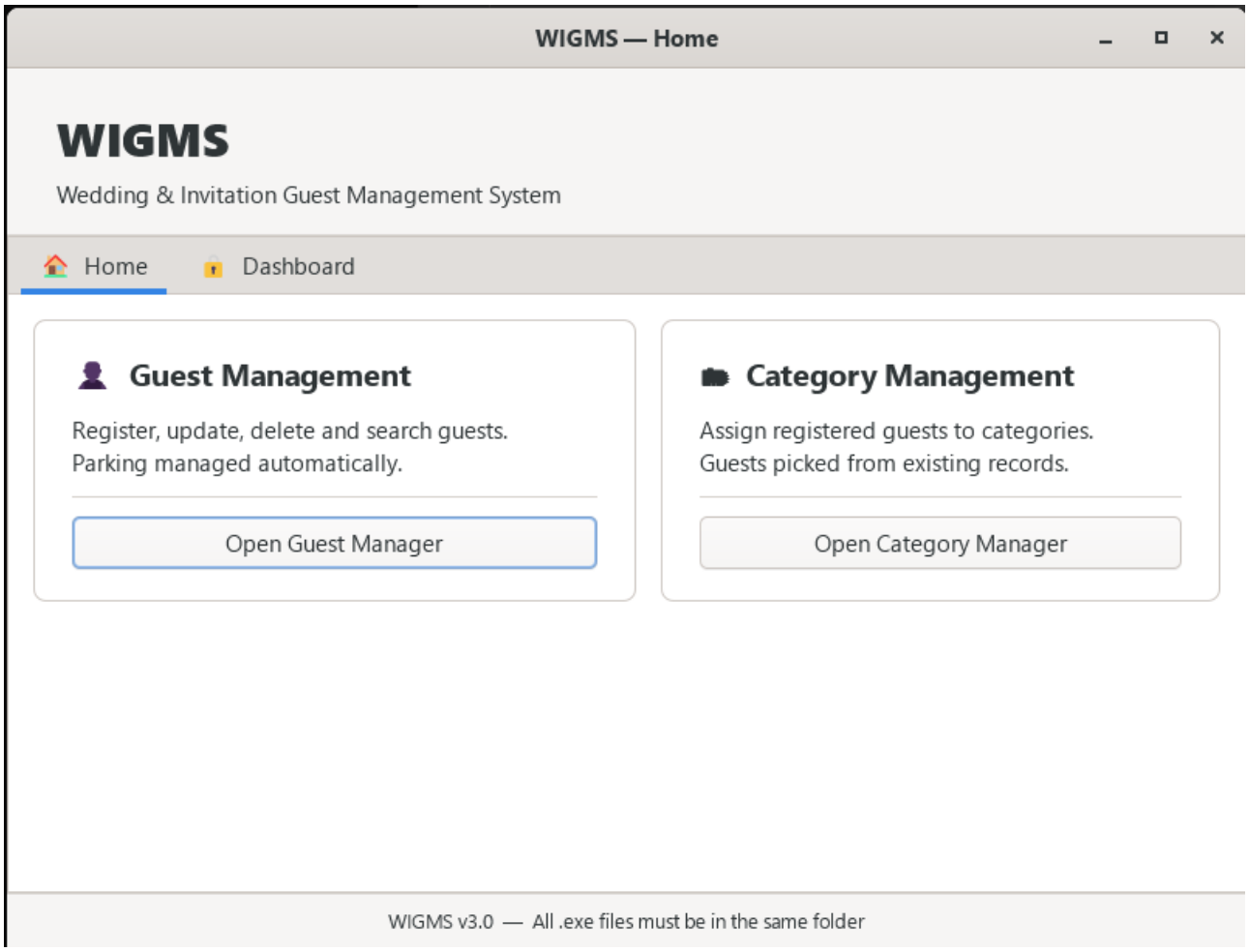
The GitHub repository was created and validated by all group members. The project can be accessed using the following link: https://github.com/C-programming-project-1/C-PROGRAMMING-PROJECT-REPOSITORY

6. Graphic User Interface (GUI's)

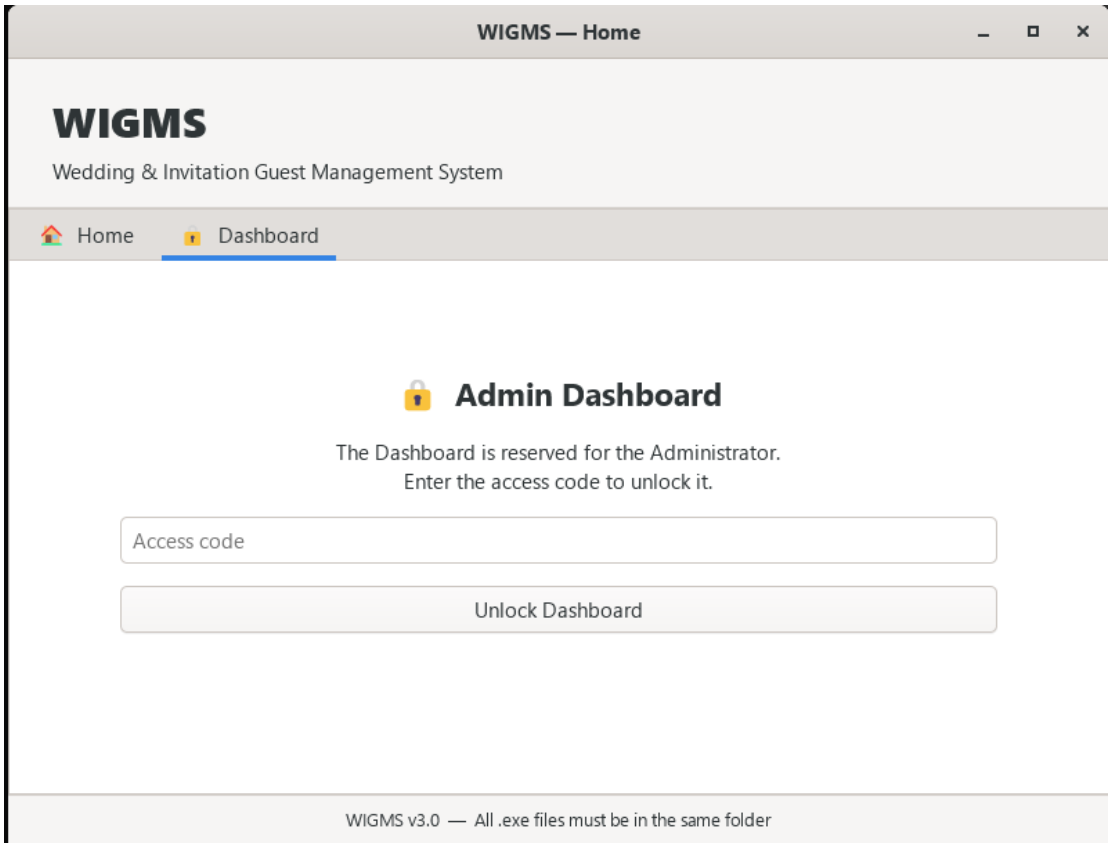
Home page

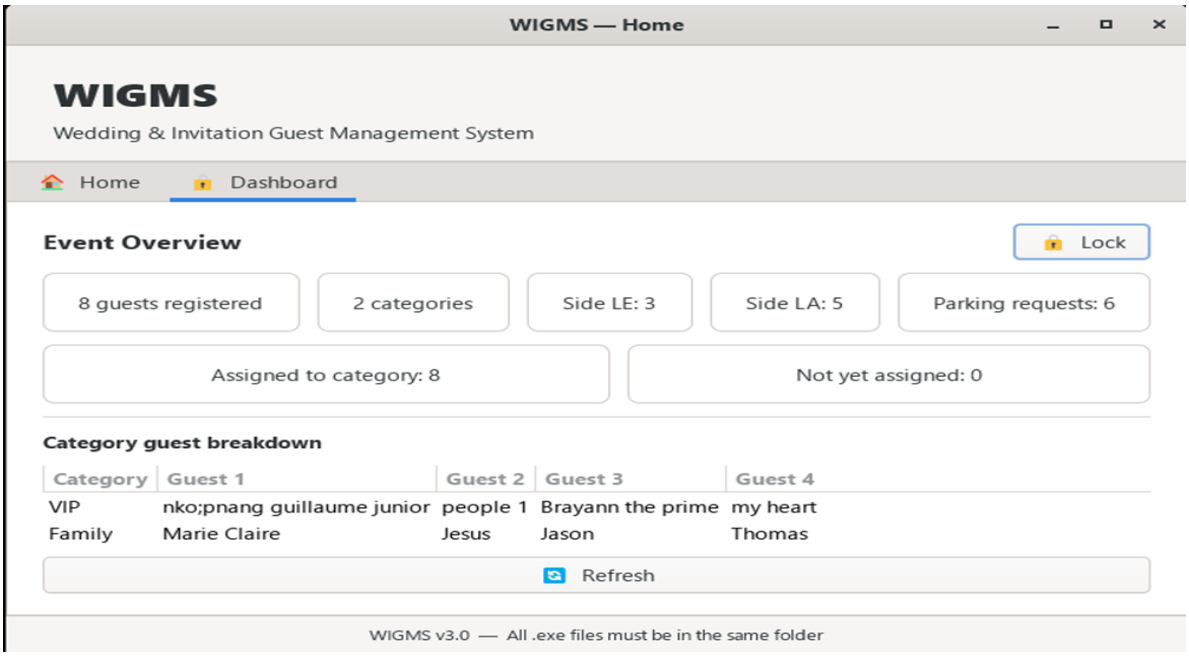
Here in this program, we can see the home page is handling two main part:

- **The Home:** that's where each user come to connect itself in order to have access to the differents functionalities of the application. Here, since we have just handled two module (and occasionally the third one); we could have access to the person/Guest management system and to the category management system.



- **The Dashboard:** Here, we are handling a general presentation of all the data which are manage by the application through a sort of recapitulative page in order to let the admin of the application have a general view of the data mange by the app. Since the dashboard is a page where the data of all the people are presented, we think that this part should be confidential and be exclusively accessible to the Admin of the app. That's why to access to this page, the user should know the access key of the dashboard, which is "**pokemon 1**". After entering the access key, you could have the access to all the data of the programm as you can see below

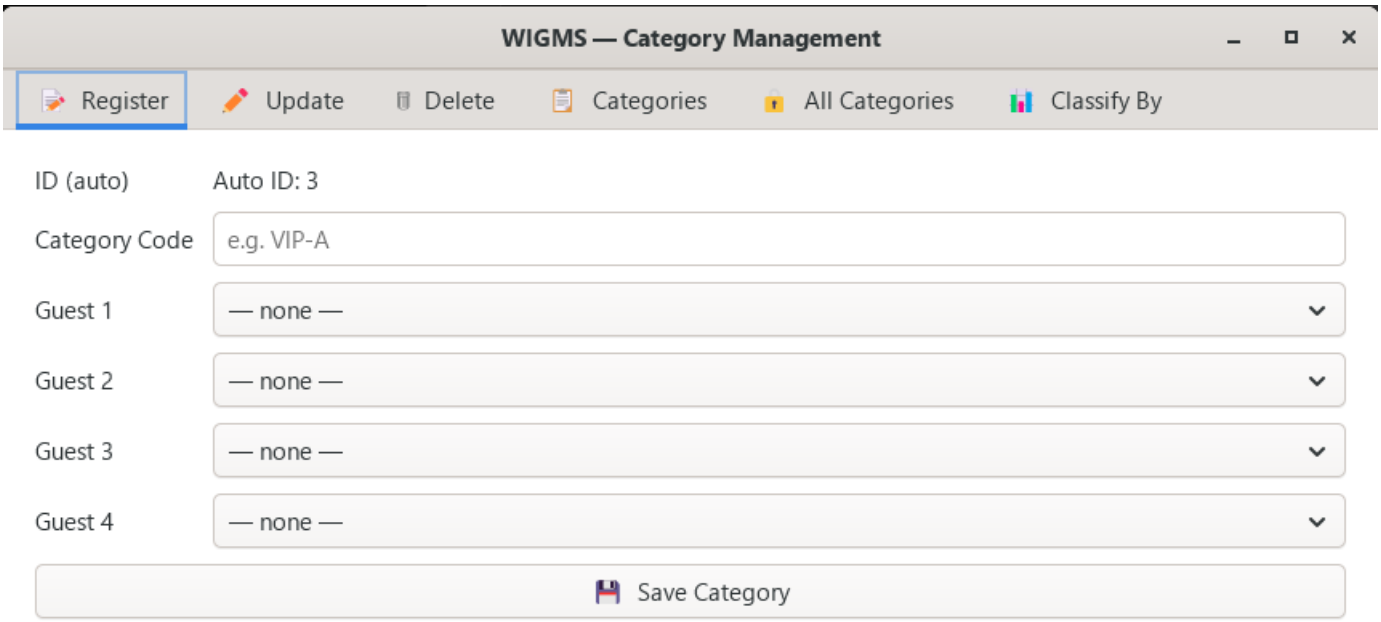


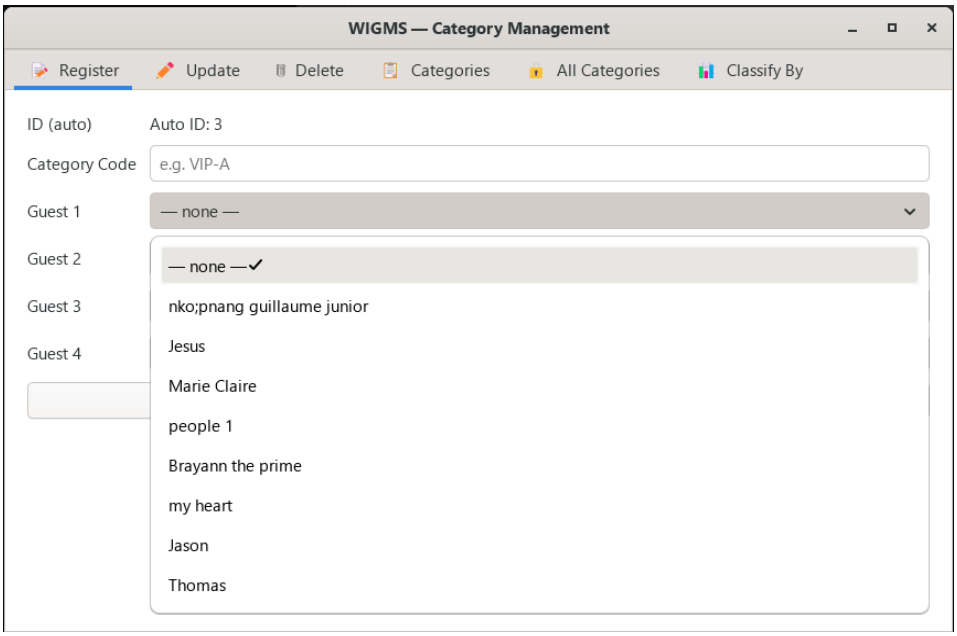


Category Module for Graphical User Interface

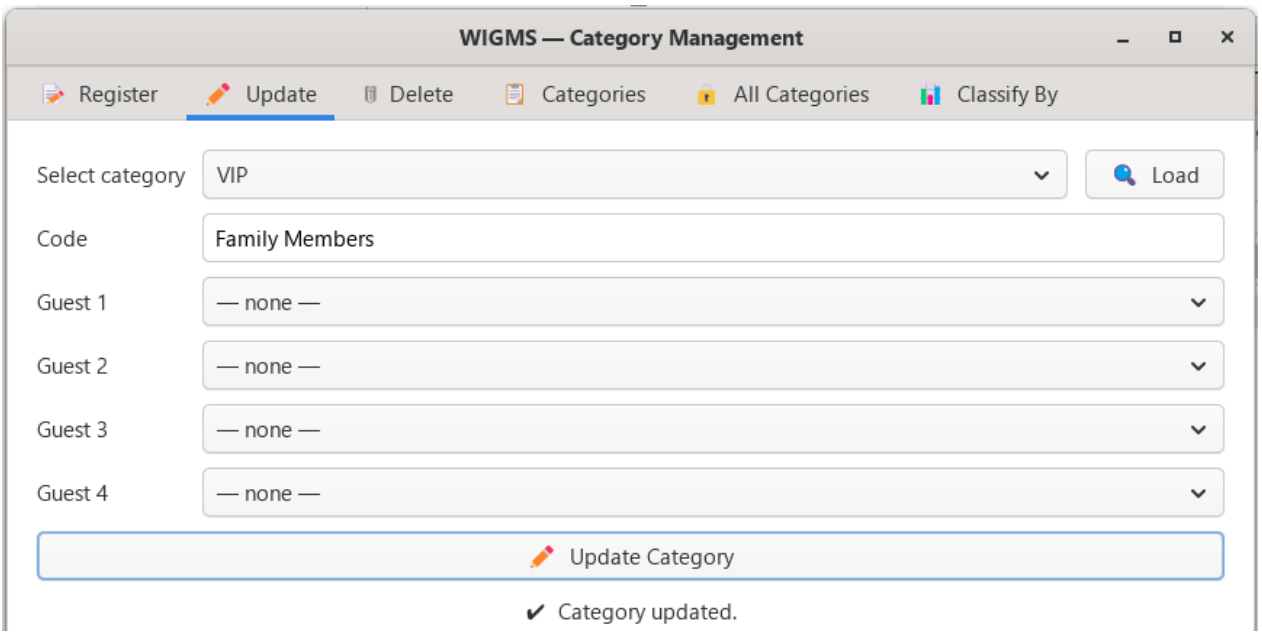
In the category module, we are handling the different kind of people which could come to register themselves: the friend, the family, the VIP guests, the partners, etc. Here, there are a lot of tasks which can be done, all subdivided into Classes. They are:

- **Register:** here, we are registering the different category of people which will be used for the organization of the ceremony. Here, we linked the person and the category module in order to enable the different guest which are already register to be automatically assign to an existent category. We also made ourselves sure to let the ID of each category to be automatically generated in order to leave redundancies and promote privacy in the program

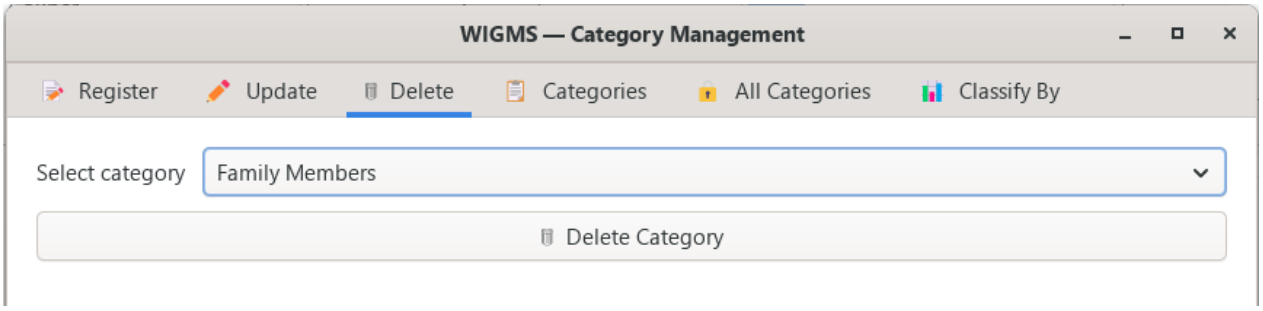




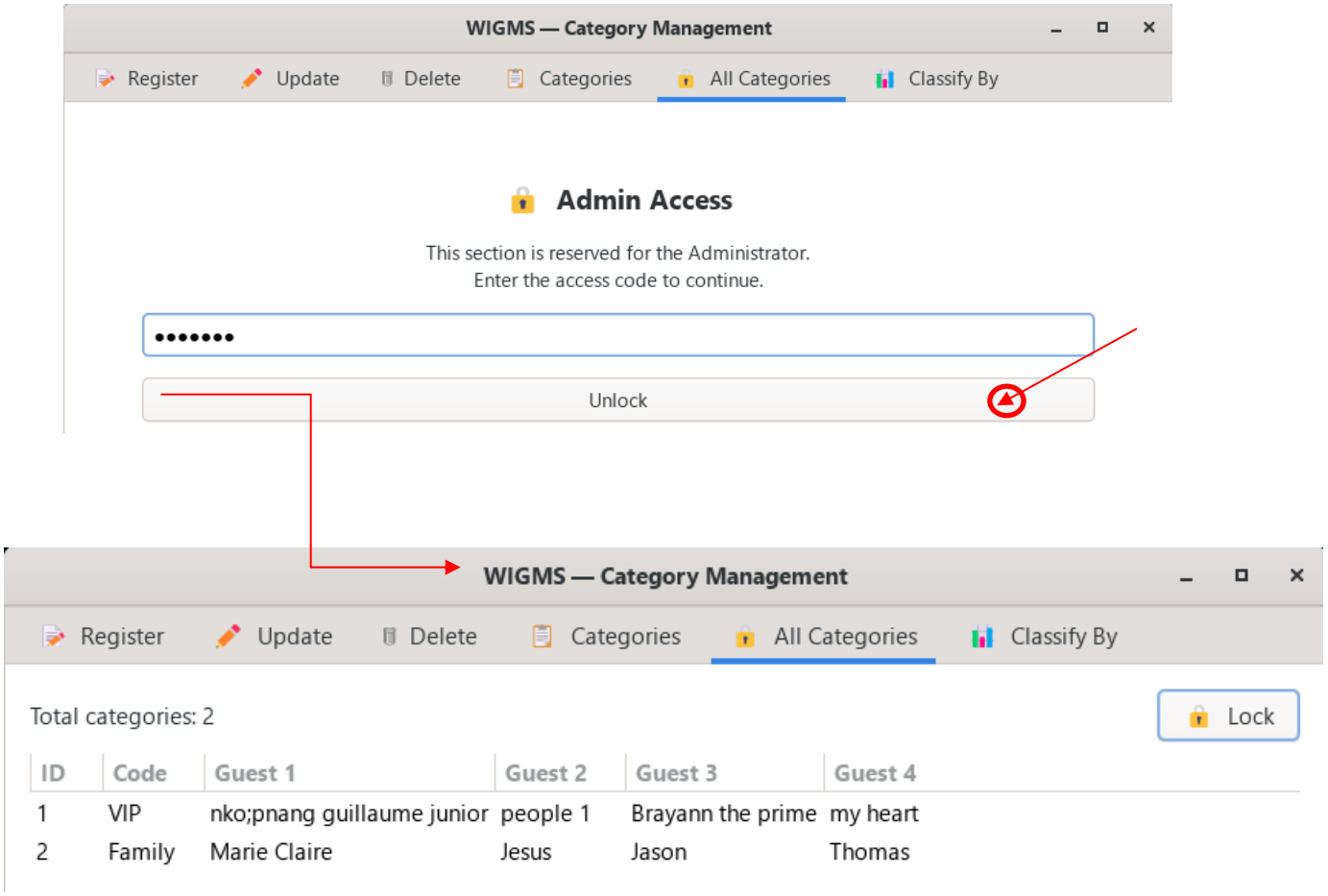
- **Update**: here we have the ability to modify the information on each category just by entering the name of the of the category in the research space and just modifying the information which are already inside the category description. Like here, we change the already existant VIP category into family members



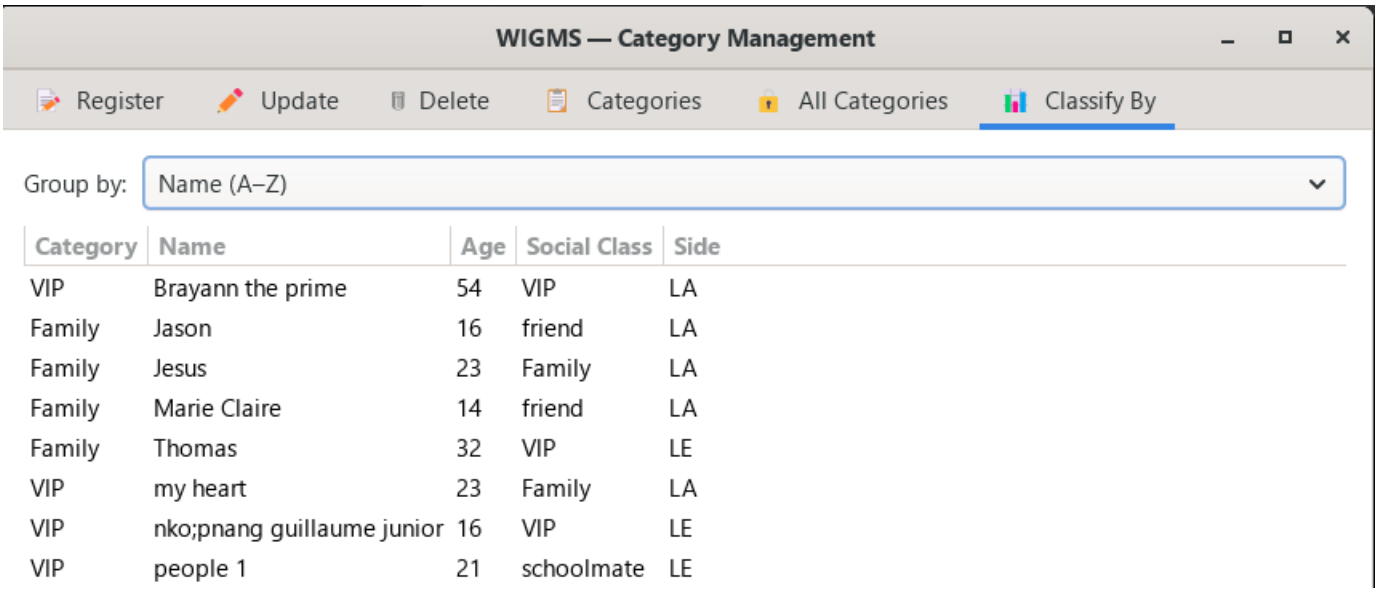
- **Delete:** It's an option where it's possible for an user to delete an existant category and the contant of these in the case that you have created an category which where not supposed to exist. It's like on this picture, where you just have the charge of write the name of the category and select delete.

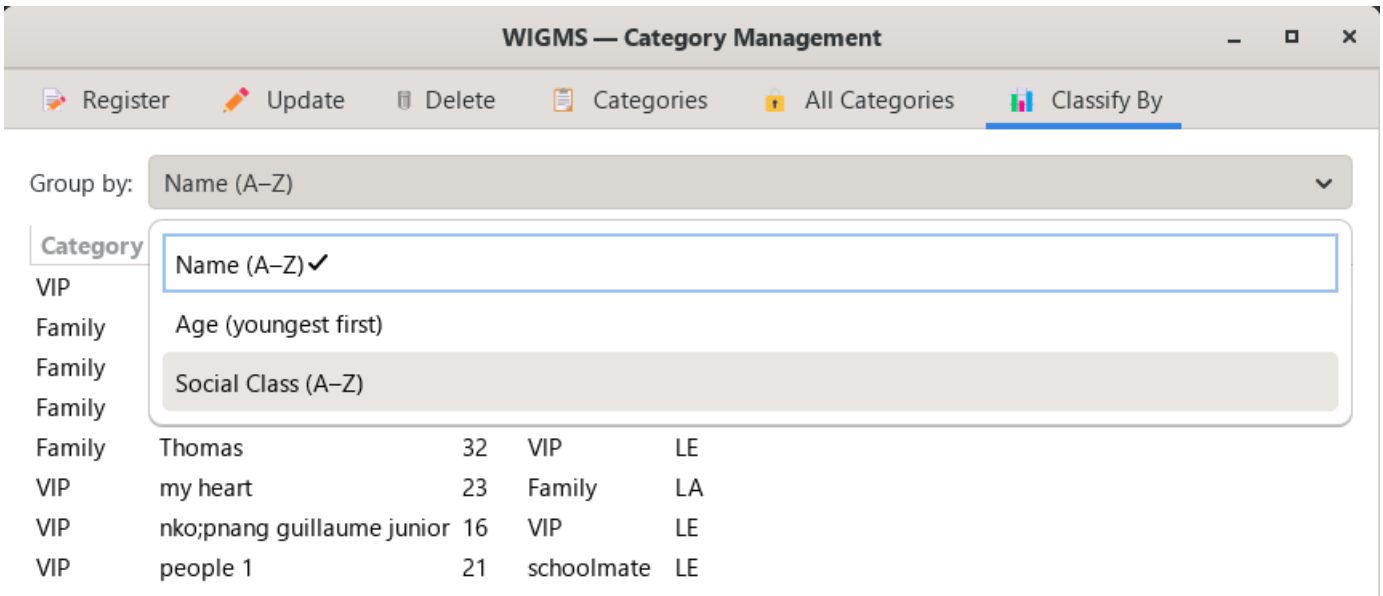


- **All categories:** this class give to a user the access to all the different categories which were generated by the program, the total number of categories, and the number of Guests which were assigned to a category, the ID of all the categories that where created in the application. Since the ID and the information about the different Guests and categories are confidential, we thought that it will be wise to protect them with a password which is “pokemon” in order to assure the protection of their data. The presentation of this page is like this:



- **Classify by:** here, we are a bit treating the functionalities of the third module. In this class, we are classify the entered by Guest according to some criteria like the name, the age or the social class in order to prioritize some category during the ceremony towards other one. It's as we can below:





7. Tools and Technologies Used

- **C Programming Language for software development.**

- **GitHub platform for version control and team collaboration.**

- **Standard C libraries such as stdio.h and string.h**

8. Difficulties Encountered and Proposed Solutions

During the development of the Wedding Invitation and Gift Management System, the group encountered several general challenges that affected both Module 1 and Module 2. These challenges were related to programming concepts, system design, and teamwork. However, appropriate solutions were applied to ensure the successful progress of the project.

8.1 Difficulty in Managing Dynamic Data Structures

The implementation of linked lists in Module 2 introduced another challenge. Unlike static data structures such as arrays, linked lists require proper handling of memory allocation and pointer connections between nodes. Errors such as incorrect linking of nodes, loss of data, and program crashes occurred during testing.

Proposed Solution:

The group used diagrams to visualize how linked lists work and how nodes are connected. Step-by-step tracing of the code was performed to identify errors. Additional practice on linked lists helped improve understanding and correct implementation.

8.2 Difficulty in Handling User Input

Managing user input using functions like scanf () was problematic, especially when handling both strings and integers. Issues such as skipped inputs and incorrect data reading were encountered.

Proposed Solution:

The group reviewed proper input handling techniques and improved the structure of input statements.

Additional validation checks were introduced to ensure correct data entry. Extensive testing was conducted to eliminate input errors.

8.3 Difficulty in Time Management and Task Coordination

Coordinating tasks among group members and meeting deadlines was sometimes difficult due to different schedules and workloads.

Proposed Solution:

Tasks were clearly assigned to each member, and deadlines were set for each phase of the project. Regular communication and progress tracking ensured that the work was completed on time.

9. Next Steps of the Project

The next phase will focus on:

- Implementing Priority Management (sorting algorithms)
- Developing Gift Management module
- Designing the User Interface
- Testing and debugging of system.

10. Conclusion

This third report highlights both the correction of Module 1 and the successful implementation of Module 2. The project has significantly improved the group's understanding of:

- Structures
- Pointers
- Linked lists
- Modular programming

Additionally, the introduction of a user-friendly interface has made the system more interactive and efficient. This module lays a strong foundation for the continuation of the project.